Project Completion Checklist

This comprehensive checklist breaks down the tasks required to fully resolve the structural, UI, and code-quality issues in the StyClick application. Completing all items on this checklist ensures a robust, maintainable, and high-performance application.

1. Architecture & State Management

Currently, the app relies heavily on `setState` and direct `SharedPreferences` reads, leading to tightly coupled UI and business logic. The goal is to fully migrate to `provider` (or another chosen state management solution) and optimize application initialization.

- [ ] **State Management Implementation (Provider)**
- [ ] Uncomment and properly configure `MultiProvider` at the root of the application (e.g., in `main.dart`).
- [ ] Create dedicated provider classes for core domains:
  - `AuthProvider`: Manage user authentication, tokens, and profile data.
  - `WalletProvider`: Manage wallet balances and transaction history.
  - `OrderProvider`: Manage cart items, saved items, and active orders.

  [ ] Refactor existing widgets (e.g., `AccountPage`, `AddFundsPage`, `WalletPage`) to consume state via `context.watch`, `context.read`, or `Consumer` instead of using local `setState` for global data.

  [ ] **Refactor Application Initialization**
- [ ] Remove the hardcoded `Future.delayed(const Duration(seconds: 5))` in `redirect.dart` (or wherever Splash/Redirect logic is executed).
- [ ] Create an `AppInitializer` service that asynchronously loads `SharedPreferences`, validates auth tokens via an API call, and pre-fetches essential data (e.g., wallet balance) concurrently.

  [ ] Use `FutureBuilder` or a dedicated Splash screen state to handle the transition to either `LoginScreen` or `Nav` (Dashboard) dynamically based on the initialization result.

  [ ] **Clean Up SharedPreferences Usage**
- [ ] Stop calling `getStringAsync` synchronously within widget `build()` methods (e.g., in `AccountPage` and `redirect.dart`).
- [ ] Load preferences once during app startup and store them in the relevant Provider (e.g., `UserProvider`), updating them reactively.

2. UI, UX & Responsiveness

The app has issues with screen scaling, non-functional UI elements, and incomplete features that degrade the user experience.

- [ ] **Fix Screen Scaling (`flutter_screenutil`)**
- [ ] Update `ScreenUtilInit` in `main.dart`. Replace `designSize: Size(logicalWidth(), logicalHeight())` with the exact dimensions used in the Figma designs (e.g., `designSize: Size(390, 844)` or `Size(375, 812)`).

  [ ] Verify that all text sizes (`.sp`), widths (`.w`), heights (`.h`), and radiuses (`.r`) scale proportionally across different device sizes without overflowing or looking distorted.

  [ ] **Implement Non-Functional Navigation**
- [ ] In `AccountPage`, implement the empty callback for **'Chats'**: `() => const ChatScreen().launch(context)` (create `ChatScreen` if it doesn't exist).
- [ ] In `AccountPage`, implement the empty callback for **'Supports / Help & Support'**: `() => const SupportPage().launch(context)` (create `SupportPage` if it doesn't exist).

  [ ] Ensure all Drawer and EndDrawer menu items map to a valid screen or action.

  [ ] **Complete Unfinished Features**
- [ ] In `add_funds.dart`, replace `// TODO: launch paymentUrl in webview/browser` with a functional payment gateway integration (e.g., Paystack, Flutterwave, or a secure WebView).
- [ ] Ensure the wallet balance updates reactively upon successful payment gateway callbacks.
- [ ] Complete any other `TODO` placeholders across the codebase.

3. Code Quality & Static Analysis

Running `flutter analyze` reports over 400 issues. These must be driven down to 0 to ensure code stability and maintainability.

- [ ] **Fix Missing `const` Constructors**

  [ ] Add `const` keywords to widgets, EdgeInsets, and other compile-time constants to optimize Flutter's widget tree rebuilding.

  [ ] **Resolve Widget Immutability Issues**
- [ ] Ensure all fields in `StatefulWidget` or `StatelessWidget` classes are marked as `final`.

  [ ] If a widget requires mutable state, properly move those variables into the corresponding `State` class.

  [ ] **Update Deprecated Flutter APIs**
- [ ] Replace deprecated theme properties (e.g., `headline6` -> `titleLarge`, `bodyText1` -> `bodyLarge`).
- [ ] Replace `WillPopScope` with `PopScope`.

  [ ] Review the `flutter analyze` log to update any other outdated Flutter 3.x APIs.

  [ ] **Clean Up Debug Statements & Unused Code**

- [ ] Remove all `print()` and `log()` statements from production code (e.g., `log('[WALLET] Funding wallet...')` in `add_funds.dart`). Use a proper logging package like `logger` that is configured to be silent in release mode.

  [ ] Remove unused imports, unused variables, and dead code blocks.

  [ ] **Enforce Linting Rules**

- [ ] Update `analysis_options.yaml` to include standard strict rules (e.g., `flutter_lints`).
- [ ] Set up a pre-commit hook or CI pipeline to run `flutter analyze` and reject code with static analysis errors.

4. Routing & Navigation Structure

The app uses raw `Navigator.push` and `Navigator.pushReplacement` throughout the application, leading to a fragmented and unmanageable navigation state.

- [ ] **Implement a Centralized Router**
- [ ] Integrate a modern routing package (e.g., `go_router` or `auto_route`) to handle deep linking, nested navigation, and route guards (e.g., auth checks).
- [ ] Refactor all `Navigator.push` calls (like in `AccountPage` and `redirect.dart`) to use named routes (e.g., `context.go('/wallet')`).
- [ ] **Route Guarding**
- [ ] Ensure that authenticated routes cannot be accessed without a valid token. Unauthenticated users should be seamlessly redirected to the login flow.

5. API & Networking Integrity

Network calls are scattered, and there is no unified approach to handling HTTP requests, headers, or authentication tokens.

- [ ] **Standardize the Networking Client**
- [ ] Centralize API calls using a robust HTTP client like `dio`.
- [ ] Implement interceptors to automatically attach the `Authorization` header (`Bearer token`) to every outgoing request.
- [ ] Implement an interceptor to handle `401 Unauthorized` responses globally (automatically logging the user out and redirecting to the login screen).
- [ ] **Centralize API Endpoints**
- [ ] Ensure all API routes are strictly defined in an `endpoints.dart` or `ApiConstants` class to avoid hardcoded strings across services (e.g., in `redirect.dart`).

6. Error Handling & Form Validations

The application lacks consistent user feedback mechanisms for failures, and forms do not thoroughly validate input before submission.

- [ ] **Global Error Handling**
- [ ] Create a centralized error handling service to catch exceptions from API calls and display standardized user-friendly `SnackBars` or Dialogs (e.g., "Network error", "Server unavailable").
- [ ] **Form Validations**
- [ ] Apply rigorous validation logic (using `FormState` and `TextFormField` validators) on login, sign up, profile edit, and vendor onboarding screens.
- [ ] Prevent form submissions (like `AddFundsPage`) if fields are empty, ensuring buttons are disabled or show appropriate inline errors instead of relying solely on `toast()`.

7. Component Reusability & Theming

The UI contains duplicated styling logic instead of leveraging Flutter's built-in `ThemeData`.

- [ ] **Standardize `ThemeData`**
- [ ] Define global text styles (using `GoogleFonts.montserrat` or `Cinta`), colors (`primary`, `cream`, `sand`), and button shapes in the `ThemeData` in `main.dart`.
- [ ] Remove hardcoded `TextStyle` and `BoxDecoration` definitions from individual widgets (like `AccountPage`) and use `Theme.of(context)` instead.
- [ ] **Refactor Reusable Widgets**
- [ ] Extract repeated UI patterns (e.g., the quick action cards in `AccountPage`, the payment method tiles in `add_funds.dart`) into distinct, reusable widgets in a shared `widgets` folder.